

Projekt architektury bezpieczeństwa dla środowiska chmurowego

Łukasz Pietrulewicz, Jakub Ratajczak

Spis treści

Wstęp	3
Charakterystyka środowiska i analiza ryzyka	4
a. Charakterystyka środowiska Pirat	4
b. Identyfikacja zasobów	4
c. Analiza zagrożeń metodą STRIDE	5
d. Macierz ryzyka	5
Architektura defense in depth	6
a. Warstwa tożsamości i dostępu	6
b. Segmentacja sieci	7
c. Szyfrowanie danych	8
d. Monitoring i wykrywanie incydentów	8
Implementacja kontroli zgodnie z regulacjami	10
a. Mapowanie wymagań na kontrole	10
b. Problem - kontrole bez dowodów	10
c. Regulacje, których nie zastosowaliśmy w pełni	11
Zarządzanie kluczami i tożsamością	12
a. Dlaczego to krytyczne?	12
b. KMS i HSM w Pirat	12
c. Envelope encryption - jak działa w praktyce	12
d. Rotacja kluczy	13
e. PKI i certyfikaty	13
f. Polityki dostępu do kluczy i audytu	13
Plan reagowania na incydenty	14
a. Cykl reagowania na incydent:	14
b. Przykładowy scenariusz	15
c. Obowiązek zgłoszenia naruszenia (RODO)	15
Część praktyczna – audyt konfiguracji bezpieczeństwa	16
a. Cel	16
b. Zasada działania	16
c. Wynik działania	17
d. Wnioski z eksperymentu	17
Wnioski i podsumowanie	18
a. Co udało się zaprojektować?	18
b. Przyjęte kompromisy	18
c. Co można rozwinąć?	18
d. Podsumowanie	19

Wstęp

W dzisiejszych czasach, coraz więcej firm przenosi swoje systemy do chmury, bo po prostu jest to tańsze i łatwiej się skaluje niż własne serwerownie. Problem jest taki, że wraz z tą migracją zmienia się też sposób myślenia o bezpieczeństwie - nie można już polegać na firewallu na granicy sieci firmowej, bo takiej granicy w chmurze właściwie nie ma.

W tym raporcie projektujemy architekturę bezpieczeństwa dla aplikacji działającej w chmurze, opartą na zasadzie defense in depth, czyli obrony warstwowej. Chodzi o to, żeby system nie padał od razu, jeśli atakujący przełamie jedno zabezpieczenie - każda warstwa powinna stanowić dodatkową przeszkodę.

Jako przykład wymyśleliśmy fikcyjną firmę Pirat - sklep internetowy, który przenosi swoją platformę sprzedażową i bazę klientów do chmury (np. AWS). System przechowuje dane osobowe użytkowników, w tym dane logowania i historię zamówień, więc trzeba też pamiętać o wymogach RODO.

Projekt obejmuje cztery główne obszary:

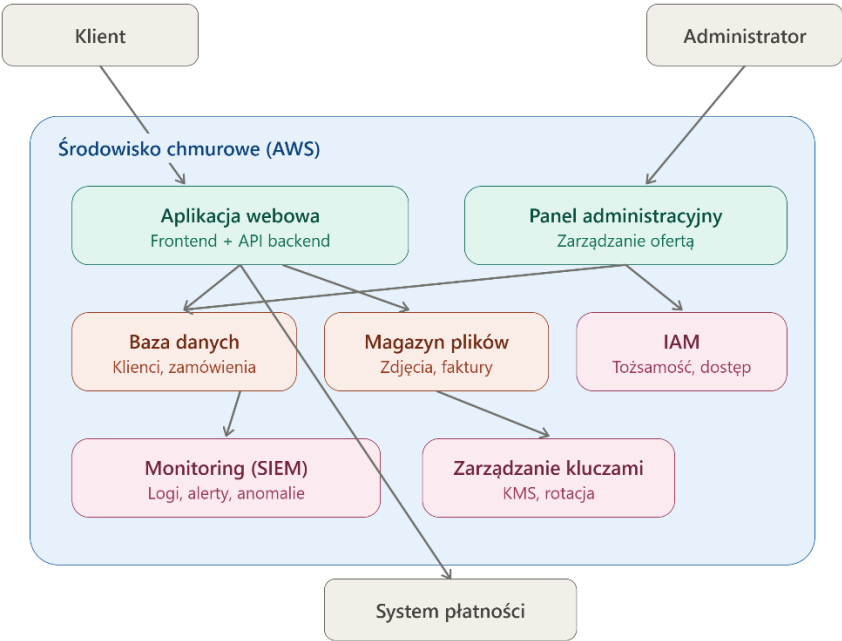
- zarządzanie tożsamością i dostępem (IAM),
- segmentację sieci,
- szyfrowanie danych,
- monitoring i wykrywanie incydentów (SIEM).

W kolejnych rozdziałach znajduje się analiza ryzyka, opis poszczególnych warstw zabezpieczeń, mapowanie kontroli do wymagań RODO/ISO 27001, kwestia zarządzania kluczami oraz plan reagowania na incydenty.

Charakterystyka środowiska i analiza ryzyka

a. Charakterystyka środowiska Pirat

Pirat to fikcyjny sklep internetowy (e-commerce), który dotychczas działał na własnych serwerach, a teraz przenosi swoją infrastrukturę do chmury AWS - czyli korzysta z serwerów, baz danych i magazynów plików wynajmowanych od Amazona, zamiast utrzymywać własną serwerownię.



Rysunek 1 Diagram architektury systemu Pirat

b. Identyfikacja zasobów

ZASÓB	WARTOŚĆ/KRYTYCZNOŚĆ
Baza danych klientów	Bardzo wysoka
Dane logowania użytkowników	Bardzo wysoka
API aplikacji	Wysoka
Panel administracyjny	Wysoka
Magazyn plików	Średnia
Logi systemowe	Średnia

c. Analiza zagrożeń metodą STRIDE

KATEGORIA	PRZYKŁAD ZAGROŻENIA
Spoofing	Podszycie się pod klienta przez przejęcie sesji/tokenu
Tampering	Modyfikacja ceny produktu w żądaniu
Repudiation	Zaprzeczenie złożeniu zamówienia
Information Disclosure	Wyciek bazy danych klientów
Denial of Service	Zalanie API żdaniami
Elevation of Privilege	Pracownik magazynu uzyskuje dostęp do panelu administracyjnego

d. Macierz ryzyka

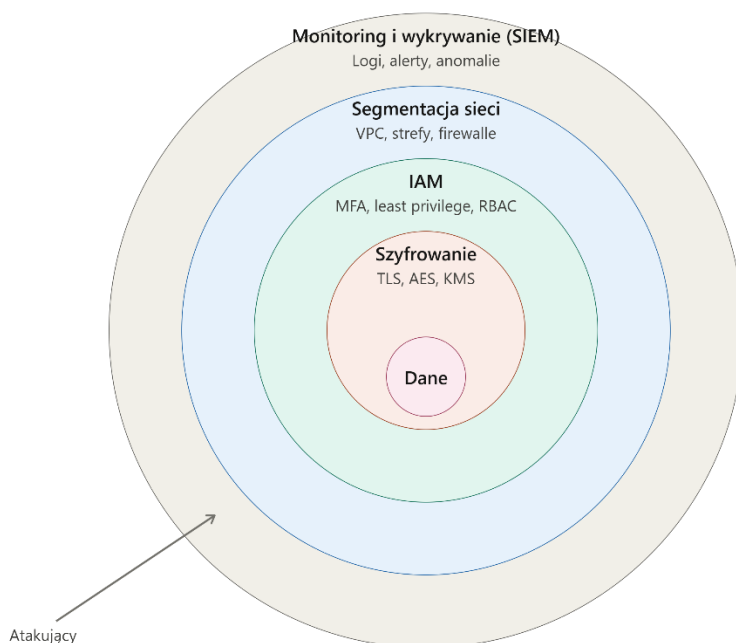
ZAGROŻENIE	PRAWDOPODOBIENSTWO	WPLYW	PRIORYTET
Wyciek bazy danych	Średnie	Bardzo wysoki	Krytyczny
Przejęcie konta klienta	Wysokie	Wysoki	Krytyczny
Atak DDoS na API	Wysokie	Średni	Wysoki
Eskalacja uprawnień w panelu zarządzającym	Niskie	Wysoki	Średni
Manipulacja ceną żądania	Niskie	Średni	Niski

Z tej macierzy wynika, że dwa obszary wymagają największej uwagi: ochrona danych przechowywanych w chmurze oraz uwierzytelnianie użytkowników. To naturalnie prowadzi do warstw architektury, które opisujemy w następnym rozdziale - przede wszystkim IAM i szyfrowanie danych.

Warto zaznaczyć, że nie każde zidentyfikowane ryzyko wymaga dedykowanego mechanizmu ochronnego. Ryzyka o niskim priorytecie, takie jak manipulacja ceną w żądaniu, mogą zostać świadomie zaakceptowane na tym etapie projektu - ich zaadresowanie nastąpi pośrednio poprzez ogólne mechanizmy walidacji danych wejściowych, bez potrzeby budowania dla nich osobnej kontroli.

Architektura defense in depth

Zgodnie z zasadą obrony warstwowej, zaprojektowaliśmy architekturę Pirat tak, aby żadna pojedyncza warstwa nie była jedynym zabezpieczeniem. Poniżej opisujemy cztery kluczowe warstwy: IAM, segmentację sieci, szyfrowanie oraz monitoring.



Rysunek 2 Schemat defense in depth

a. Warstwa tożsamości i dostępu

Zarządzanie tożsamością to pierwsza linia obrony - większość ataków na systemy chmurowe zaczyna się od przejęcia jakiegoś konta lub wykorzystania nadmiarowych uprawnień.

Zasada least privilege

Każdy komponent systemu dostaje dokładnie te uprawnienia, których potrzebuje - nic więcej. Przykładowo, aplikacja webowa ma prawo odczytu i zapisu do bazy danych zamówień, ale nie ma uprawnień do usuwania całych tabel czy modyfikacji uprawnień innych użytkowników.

Role serwisowe vs konta użytkowników:

- **Role IAM dla usług** – np. aplikacja backend łącząca się z bazą danych działa jako rola serwisowa z konkretnymi, wąskimi uprawnieniami
- **Konta użytkowników** – pracownicy logujący się do panelu administracyjnego, z obowiązkową weryfikacją wieloetapową

MFA dla kont administracyjnych

Dla pracowników z dostępem do panelu administracyjnego MFA jest obowiązkowe i niemożliwe do wyłączenia. Wynika to z analizy ryzyka - przejęcie konta administratora ma dużo bardziej znaczący wpływ niż przejęcie konta klienta.

Polityki dostępu oparte na rolach

Stosujemy RBAC do rozdzielenia uprawnień w panelu administracyjnym - pracownik magazynu ma dostęp tylko do modułu zamówień i stanów magazynowych, a nie np. do raportów finansowych czy ustawień konta.

b. Segmentacja sieci

Podział na strefy (VPC i podsieci):

- **Strefa publiczna** - tylko load balancer i serwery frontendowe, dostępne z internetu
- **Strefa prywatna** - serwery aplikacyjne (API), niedostępne bezpośrednio z internetu
- **Strefa danych** - baza danych, dostępna wyłącznie z serwerów aplikacyjnych

Odseparowanie panelu administracyjnego

Panel administracyjny nie jest dostępny z publicznego internetu na takich samych zasadach jak sklep dla klientów. Dostęp do niego wymaga połączenia przez VPN lub jest ograniczony do konkretnych adresów IP (np. sieci biurowej firmy).

Firewalle i grupy bezpieczeństwa

Każda strefa ma zdefiniowane reguły ruchu (security groups) - np. baza danych przyjmuje połączenia tylko z serwerów aplikacyjnych na konkretnym porcie, a nie z całego internetu.

Ograniczenie lateral movement

Nawet jeśli atakujący przejmie jeden komponent (np. serwer frontendowy), segmentacja utrudnia mu przesunięcie się dalej do bazy danych czy panelu administracyjnego.

c. Szyfrowanie danych

Szyfrowanie w tranzycie

Cała komunikacja między klientem a aplikacją oraz między komponentami wewnątrz chmury odbywa się przez TLS/HTTPS - zarówno żeby chronić dane klientów w drodze, jak i żeby zapobiec atakom typu man-in-the-middle.

Szyfrowanie w spoczynku

- Baza danych jest szyfrowana (encryption at rest) - nawet jeśli ktoś uzyska nieautoryzowany dostęp do dysku, dane są nieczytelne bez kluczy
- Magazyn plików (zdjęcia, faktury) jest również szyfrowany
- Szczególnie chronione są dane logowania - hasła nigdy nie są przechowywane w postaci czystego tekstu, tylko jako hashe (np. bcrypt/Argon2)

Envelope encryption

W praktyce stosujemy podejście envelope encryption - dane użytkownika są szyfrowane kluczem symetrycznym, a ten klucz jest dodatkowo szyfrowany kluczem głównym przechowywanym w usłudze zarządzania kluczami (KMS).

d. Monitoring i wykrywanie incydentów

Centralizacja logów (SIEM)

Logi z aplikacji, bazy danych, panelu administracyjnego oraz infrastruktury sieciowej są zbierane centralnie. Dzięki temu można skorelować zdarzenia z różnych źródeł - np. nieudane logowania w aplikacji z podejrzanym ruchem sieciowym w tym samym czasie.

Wykrywanie anomalii:

- Wielokrotne nieudane logowania do panelu administracyjnego (możliwy brute force)
- Logowanie admina z nieznanej lokalizacji
- Nagły wzrost liczby zamówień z jednego adresu IP

Alerty i reakcja

Wykryte anomalie generują alerty dla zespołu odpowiedzialnego za bezpieczeństwo

Jak warstwy wzajemnie się wspierają?

Żadna z czterech warstw nie działa w izolacji - to jest właśnie sens defense in depth. Przykładowo: jeśli atakujący zdobędzie hasło administratora przez phishing (przełamanie pierwszej warstwy — uwierzytelniania), to:

- MFA blokuje go, mimo że ma hasło (wciąż warstwa IAM)
- Jeśli MFA też zostanie przełamane, segmentacja sieci ogranicza, do czego może się dostać
- Jeśli mimo to uzyska dostęp do danych, szyfrowanie chroni dane, do których nie ma kluczy
- Monitoring wykrywa nietypowe logowanie i generuje alert, nawet jeśli wcześniejsze warstwy zawiodły

Implementacja kontroli zgodnie z regulacjami

Sama architektura defense in depth, choć dobrze zaprojektowana, nie wystarczy - trzeba też umieć wykazać, że spełnia się konkretne wymogi regulacyjne. W przypadku Pirat dotyczy to przede wszystkim RODO (bo przetwarzamy dane osobowe klientów) oraz, jeśli firma rośnie i staje się większym podmiotem, ISO/IEC 27001 jako standardu zarządzania bezpieczeństwem informacji.

a. Mapowanie wymagań na kontrole

WYMAGANIE	KONTROLA W PIRAT	DOWÓD DZIAŁANIA
RODO – minimalizacja danych	Aplikacja zbiera tylko niezbędne dane do realizacji zamówienia	Dokumentacja modelu danych
RODO – integralność i poufność danych	Szyfrowanie w tranzycie i spoczynku	Konfiguracja KMS i certyfikaty TLS
RODO – zgłoszenie naruszenia w 72h	Monitoring SIEM wykrywający anomalie	Logi z SIEM, historia alertów
RODO – prawo do dostępu/usunięcia danych	Funkcjonalność w panelu klienta do eksportu/usunięcia konta	Logi operacyjne na koncie użytkownika
ISO 27001 – kontrola dostępu	IAM z least privilege, MFA, RBAC	Polityki IAM, rejestr ról i uprawnień
ISO 27001 – kryptografia	Szyfrowanie danych, zarządzanie kluczami (KMS)	Polityka zarządzania kluczami, harmonogram rotacji
ISO 27001 – ciągłość działania	Backup i replikacja danych	Raporty z testów odzyskiwania

b. Problem - kontrole bez dowodów

Najczęstszym błędem organizacji jest to, że kontrole formalnie istnieją, ale nie ma dowodów ich działania. W przypadku Pirat staramy się tego uniknąć - każda kontrola z tabeli powyżej ma przypisany konkretny dowód (log, konfigurację, raport), a nie jest tylko deklaracją "u nas to działa".

Przykładowo, samo stwierdzenie "stosujemy MFA" nie wystarczy dla audytora - potrzebny jest dowód, np. zrzut konfiguracji polityki IAM wymuszającej MFA dla wszystkich kont administracyjnych, albo raport z logów pokazujący, że logowania bez drugiego czynnika są odrzucane.

c. Regulacje, których nie zastosowaliśmy w pełni

- **PCI-DSS** - Pirat nie przechowuje bezpośrednio danych kart płatniczych (korzysta z zewnętrznego dostawcy płatności, np. Przelewy24), więc nie podlega pełnym, twardym wymaganiom technicznym tego standardu. Odpowiedzialność za bezpieczeństwo samych danych karty przechodzi na dostawcę płatności.
- **NIS2** - dotyczy głównie operatorów usług kluczowych i dużych dostawców usług cyfrowych. Pirat jako mały/średni sklep internetowy nie jest objęty tym obowiązkiem, ale przy dalszym skalowaniu działalności warto to monitorować.

Zarządzanie kluczami i tożsamością

Ten rozdział rozwija temat poruszony już częściowo w rozdziale 3, ale skupiamy się tutaj na szczegółach technicznych zarządzania kluczami kryptograficznymi, bo to osobny i często niedoceniany element bezpieczeństwa.

a. Dlaczego to krytyczne?

Samo szyfrowanie danych nic nie daje, jeśli klucze są źle zarządzane - przechowywane w kodzie aplikacji, w plikach konfiguracyjnych .env trzymany w repozytorium, albo nigdy nie rotowane.

b. KMS i HSM w Pirat

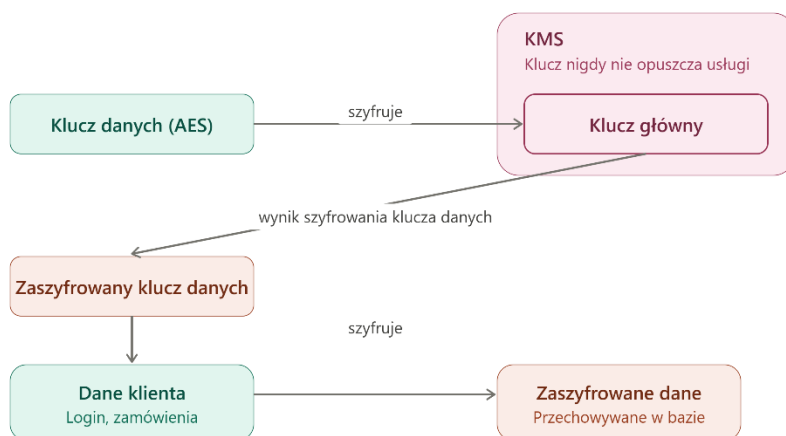
Zamiast przechowywać klucze szyfrujące samodzielnie w kodzie czy bazie danych, korzystamy z usługi KMS (Key Management Service) dostępnej w chmurze AWS. KMS to bezpieczne, zarządzane repozytorium kluczy, które:

- generuje i przechowuje klucze główne w sposób odizolowany od aplikacji
- udostępnia API do szyfrowania/deszyfrowania bez ujawniania samego klucza aplikacji
- może być wsparte sprzętowo przez HSM (Hardware Security Module) dla najwyższego poziomu bezpieczeństwa

c. Envelope encryption - jak działa w praktyce

1. Dane klienta są szyfrowane kluczem danych (data key) - kluczem symetrycznym (AES)
2. Klucz danych jest następnie szyfrowany kluczem głównym (master key), który nigdy nie opuszcza KMS
3. W bazie danych przechowujemy dane i zaszyfrowany klucz danych, a nie sam klucz główny

Taki schemat ma istotną zaletę: jeśli trzeba zrotować klucz główny, nie musimy ponownie szyfrować wszystkich danych, a jedynie wystarczy ponownie zaszyfrować tylko klucze danych.



Rysunek 3 Schemat envelope encryption

d. Rotacja kluczy

Klucze kryptograficzne nie powinny być używane bezterminowo. W architekturze Pirat przyjęliśmy zasadę:

- klucze główne w KMS są rotowane automatycznie (np. raz w roku, zgodnie z domyślną polityką AWS KMS)
- klucze sesji (np. tokeny JWT) mają znacznie krótszy czas życia - wygasają po kilku godzinach, zmuszając użytkownika do ponownego uwierzytelnienia

Krótszy okres ważności klucza zmniejsza ryzyko - nawet jeśli klucz zostanie przechwycony, czas, w którym może być wykorzystany przez atakującego, jest ograniczony.

e. PKI i certyfikaty

Komunikacja TLS wymaga infrastruktury PKI (Public Key Infrastructure) - certyfikatów wydanych przez zaufane CA (Certificate Authority), które potwierdzają, że serwer Pirat jest tym, za kogo się podaje. W praktyce certyfikaty są zarządzane automatycznie przez usługę chmurową (np. AWS Certificate Manager), co eliminuje ryzyko zapomnienia o odnowieniu certyfikatu.

f. Polityki dostępu do kluczy i audytu

Dostęp do operacji na kluczach (np. żądanie odszyfrowania danych) jest ograniczony politykami IAM - tylko konkretne role serwisowe (np. backend aplikacji) mogą wywołać operację deszyfrowania, a każde takie wywołanie jest logowane. Dzięki temu, w razie incydentu, można sprawdzić, i kiedy odwoływał się do danych zaszyfrowanych kluczem.

Plan reagowania na incydenty

Nawet najlepiej zaprojektowana architektura defense in depth nie gwarantuje, że atak się nie wydarzy - jak mówi zasada „assume breach”, trzeba projektować system zakładając, że incydent w jakimś momencie nastąpi. Dlatego równie ważne jak zapobieganie jest to, co robimy, gdy mechanizmy obronne zawiodą i incydent faktycznie wystąpi.

a. Cykl reagowania na incydent:

1. Detekcja (detection)

System SIEM wykrywa anomalię, np. wielokrotne nieudane logowania do panelu administracyjnego w krótkim czasie, albo nietypowy ruch wychodzący z bazy danych. Generowany jest alert dla zespołu bezpieczeństwa.

2. Powstrzymanie (containment)

Celem jest zatrzymanie rozprzestrzeniania się incydentu, zanim zrozumiemy go w pełni. Dla Pirat oznacza to np. tymczasowe zablokowanie konta, odcięcie dostępu sieciowego do skompromitowanego elementu, unieważnienie aktywnych tokenów sesji.

3. Analiza (analysis)

Zespół sprawdza logi z SIEM, żeby ustalić zakres incydentu - które dane zostały dotknięte, jak długo trwał nieautoryzowany dostęp, jaką metodą atakujący dostał się do systemu. To etap, w którym kluczowe są dowody zbierane automatycznie (logi z IAM, logi dostępu do KMS).

4. Usunięcie przyczyny (eradication)

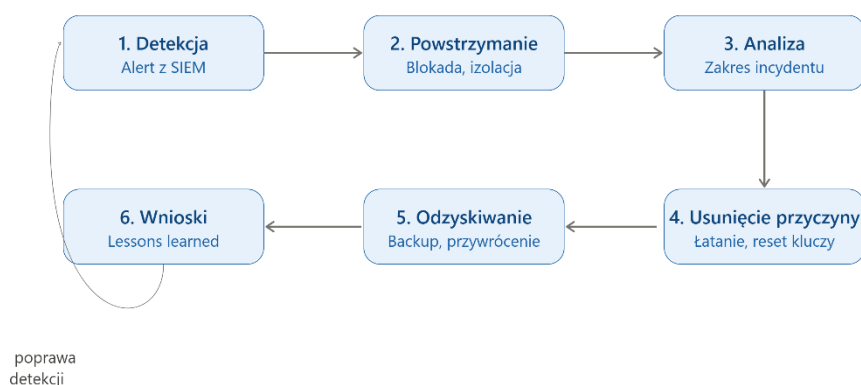
Usuwanie źródła problemu - np. łatamy podatność, która umożliwiła atak, resetujemy skompromitowane hasła/klucze, usuwamy złośliwy kod, jeśli został wprowadzony.

5. Odzyskiwanie (recovery)

Przywracamy normalne działanie systemu. Jeśli dane zostały zmodyfikowane lub zaszyfrowane, korzystamy z backupów - pod warunkiem, że są one regularnie testowane, a nie tylko istnieją.

6. Wnioski (lessons learned)

Po zamknięciu incydentu zespół analizuje, co poszło nie tak i jak zapobiec podobnym sytuacjom w przyszłości np. czy potrzebna jest dodatkowa kontrola, czy proces wykrywania był wystarczająco szybki.



Rysunek 4 Cykl reagowania na incydent

b. Przykładowy scenariusz

Przedstawimy mechanizm obronny na przykładzie przejęcia konta klienta przez słabe hasło i brak MFA:

1. **Detekcja** – SIEM wykrywa logowanie z konta klienta z nietypowej lokalizacji geograficznej, krótko po nieudanych próbach logowania
2. **Powstrzymanie** – system automatycznie wymusza wylogowanie i blokuje bieżącą sesję
3. **Analiza** – sprawdzamy, czy z konta wykonano jakieś działania np. zmiana adresu dostawy
4. **Usunięcie przyczyny** – wymuszamy reset hasła, informujemy klienta i sugerujemy włączenie MFA
5. **Odzyskiwanie** – jeśli złożono nieautoryzowane zamówienie, to jest ono anulowane
6. **Wnioski** – może być to sygnał, że MFA dla klientów powinno być obowiązkowe

c. Obowiązek zgłoszenia naruszenia (RODO)

Jak wspomnieliśmy wcześniej, RODO nakłada obowiązek zgłoszenia naruszenia ochrony danych osobowych w ciągu **72 godzin** od jego wykrycia. Nasz plan reagowania wspiera ten wymóg dzięki scentralizowanym logom z SIEM, zespół jest w stanie szybko ustalić zakres naruszenia (które dane, ilu klientów) i podjąć decyzję o zgłoszeniu do odpowiedniego organu nadzorczego.

Część praktyczna – audyt konfiguracji bezpieczeństwa

a. Cel

Architektura zaprojektowana w poprzednich rozdziałach opisuje, jak powinien wyglądać bezpieczny system Pirat. W praktyce jednak konfiguracja infrastruktury może odbiegać od projektu - przez błąd ludzki, pośpiech przy wdrożeniu albo zmianę wprowadzoną "tymczasowo", o której później ktoś zapomniał. Żeby to zweryfikować, napisaliśmy prosty skrypt w Pythonie, który automatycznie sprawdza wybrany plik konfiguracyjny pod kątem zgodności z zasadami opisanymi w rozdziale 3 - czyli least privilege, szyfrowania, MFA i segmentacji sieci.

To podejście odpowiada bezpośrednio na problem opisany w rozdziale 4.3 - sam fakt, że kontrola "powinna" działać, nie znaczy, że faktycznie działa. Skrypt daje nam dowód, a nie tylko deklarację.

b. Zasada działania

Skrypt wczytuje plik konfiguracyjny w formacie JSON, reprezentujący ustawienia infrastruktury Pirat (magazyny plików, role IAM, konta administracyjne, bazy danych, ustawienia sieciowe), i sprawdza go względem zestawu reguł. Każda reguła odpowiada konkretnemu mechanizmowi z rozdziału 3. Przykładowo, funkcja sprawdzająca role IAM wygląda następująco:

```
def sprawdz_role_iam(config, problemy):
    for rola in config.get("role_iam", []):
        nazwa = rola["nazwa"]
        akcje = rola.get("akcje", [])
        zasoby = rola.get("zasoby", [])

        if "*" in akcje and "*" in zasoby:
            problemy.append({
                "priorytet": KRYTYCZNY,
                "obszar": "IAM",
                "opis": f"Rola '{nazwa}' ma nieograniczone uprawnienia.",
                "rekomendacja": "Ograniczyć rolę do konkretnych akcji i zasobów (least privilege)."
            })
        elif "*" in akcje:
            problemy.append({
                "priorytet": WYSOKI,
                "obszar": "IAM",
                "opis": f"Rola '{nazwa}' ma dostęp do wszystkich akcji.",
                "rekomendacja": "Sprecyzować listę dozwolonych akcji."
            })
```

Rysunek 5 Funkcja sprawdzająca role IAM

Podobne funkcje sprawdzają magazyny plików (publiczny dostęp, szyfrowanie), konta administracyjne (MFA), bazy danych (szyfrowanie w spoczynku) oraz segmentację sieci (dostępność panelu administracyjnego z internetu). Każdy wykryty problem otrzymuje priorytet - krytyczny, wysoki lub średni.

c. Wynik działania

```
=====
RAPORT AUDYTU BEZPIECZEŃSTWA - Pirat
=====

Podsumowanie:
  KRYTYCZNY: 4
  WYSOKI: 3
  SREDNI: 0

Szczegóły:
-----
[KRYTYCZNY] Magazyn plików
  Problem:      Magazyn 'pirat-faktury-klientow' jest publicznie dostępny.
  Rekomendacja: Wyłączyć publiczny dostęp, ograniczyć go do roli aplikacji.

[KRYTYCZNY] IAM
  Problem:      Rola 'rola-skrypt-backup' ma nieograniczone uprawnienia.
  Rekomendacja: Ograniczyć rolę do konkretnych akcji i zasobów (least privilege).

[KRYTYCZNY] Uwierzytelnianie
  Problem:      Konto administracyjne 'admin_anna' nie ma włączonego MFA.
  Rekomendacja: Wymusić MFA dla wszystkich kont z dostępem do panelu administracyjnego.

[KRYTYCZNY] Uwierzytelnianie
  Problem:      Konto administracyjne 'admin_serwis' nie ma włączonego MFA.
  Rekomendacja: Wymusić MFA dla wszystkich kont z dostępem do panelu administracyjnego.

[WYSOKI] Magazyn plików
  Problem:      Magazyn 'pirat-faktury-klientow' nie ma włączonego szyfrowania.
  Rekomendacja: Włączyć szyfrowanie w spoczynku.

[WYSOKI] Baza danych
  Problem:      Baza 'baza-zamowienia' nie ma włączonego szyfrowania w spoczynku.
  Rekomendacja: Włączyć szyfrowanie (envelope encryption).

[WYSOKI] Segmentacja sieci
  Problem:      Panel administracyjny jest dostępny z internetu.
  Rekomendacja: Ograniczyć dostęp do VPN lub konkretnych adresów IP.
```

Rysunek 6 Raport audytu bezpieczeństwa

d. Wnioski z eksperymentu

Wynik audytu pokazuje coś istotnego: nawet dobrze zaprojektowana architektura nie obroni się sama, jeśli jej wdrożenie odbiega od projektu. W naszej testowej konfiguracji skrypt wykrył m.in. magazyn z fakturami klientów dostępny publicznie z internetu oraz rolę serwisową z nieograniczonymi uprawnieniami - czyli właśnie te scenariusze, które zidentyfikowaliśmy jako najbardziej ryzykowne w analizie STRIDE.

To pokazuje praktyczną wartość automatycznych audytów konfiguracji - w realnym środowisku chmurowym takie sprawdzenia są zazwyczaj uruchamiane regularnie (np. w ramach pipeline CI/CD lub jako część monitoringu, opisanego w rozdziale 3.4), a nie wykonywane jednorazowo "przy okazji audytu", co jak wspominaliśmy jest jednym z najczęstszych błędów organizacyjnych w podejściu do zgodności.

LINK DO EKSPERYMENTU:

https://github.com/LUKASZCOM/Audyt_bezpieczenstwa_chmurowego

Wnioski i podsumowanie

a. Co udało się zaprojektować?

W ramach projektu zaprojektowaliśmy architekturę obejmującą cztery warstwy obrony - IAM, segmentację sieci, szyfrowanie oraz monitoring - i pokazaliśmy, jak te warstwy wzajemnie się wspierają, gdy jedna z nich zawiedzie. Dodatkowo zmapowaliśmy zaprojektowane kontrole do konkretnych wymagań regulacyjnych (RODO, ISO 27001), żeby architektura nie była tylko teoretycznym ćwiczeniem, ale realnie odpowiadała na obowiązki prawne, jakie miałyby firma Pirat. Opisaliśmy też, jak wygląda zarządzanie kluczami w praktyce oraz jak zespół powinien reagować, gdy mechanizmy obronne zawiodą i dojdzie do incydentu.

b. Przyjęte kompromisy

Projektując ten system, musieliśmy podjąć kilka decyzji będących kompromisem między bezpieczeństwem, kosztem i wygodą:

- **MFA obowiązkowe dla administratorów, opcjonalne dla klientów** - pełne wymuszenie MFA dla wszystkich klientów zwiększyłoby bezpieczeństwo, ale mogłoby zniechęcić część użytkowników do korzystania ze sklepu. Dla kont administracyjnych, gdzie wpływ przejęcia konta jest znacznie wyższy, MFA jest wymagane bez wyjątków.
- **Akceptacja niektórych ryzyk niskiego priorytetu** - ryzyka takie jak manipulacja ceną w żądaniu nie wymagały dedykowanego mechanizmu, bo są pokryte ogólną walidacją danych wejściowych.
- **Brak pełnej zgodności z PCI-DSS** - zamiast budować własną, kosztowną infrastrukturę zgodną z tym standardem, Pirat przenosi to ryzyko na zewnętrznego dostawcę płatności, co jest częstym i racjonalnym rozwiązaniem dla małych/średnich firm e-commerce.

c. Co można rozwinąć?

Przy większym budżecie i skali działania, architekturę Pirat dałoby się rozszerzyć o:

- Zero-trust network access (ZTNA) zamiast tradycyjnego VPN dla dostępu administracyjnego
- Automatyczne testy penetracyjne zintegrowane z CI/CD, a nie tylko okazjonalne audyty
- Rozszerzony monitoring z UEBA (analiza zachowań użytkowników) do wykrywania bardziej subtelnych anomalii, niż tylko proste reguły
- Pełną zgodność z NIS2, jeśli firma urośnie do skali, w której zostanie uznana za istotny podmiot cyfrowy

d. Podsumowanie

Projekt ten pokazał, że bezpieczeństwo systemu informacyjnego nie jest stanem, który się "osiąga" raz i na zawsze, a ciągłym procesem podejmowania decyzji, oceny ryzyka i kompromisów. Architektura defense in depth, którą zaprojektowaliśmy dla Pirat, nie eliminuje wszystkich zagrożeń, ale też nie taki był jej cel. Jej zadaniem jest maksymalne utrudnienie ataku i ograniczenie jego skutków, nawet jeśli pojedyncze zabezpieczenie zostanie przełamane.

Spis Rysunków:

Rysunek 1 Diagram architektury systemu Pirat	4
Rysunek 2 Schemat defense in depth	6
Rysunek 3 Schemat envelope encryption	12
Rysunek 4 Cykl reagowania na incydent	14

Bibliografia:

- Amazon Web Services. *AWS Key Management Service (KMS) - Documentation*. <https://docs.aws.amazon.com/kms/>
- Amazon Web Services. *AWS Identity and Access Management (IAM) - Documentation*. <https://docs.aws.amazon.com/iam/>
- Amazon Web Services. *Security Best Practices in IAM*. <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
- OWASP Foundation. *OWASP Top 10*. <https://owasp.org/www-project-top-ten/>
- Urząd Ochrony Danych Osobowych. *Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2016/679 (RODO)*. <https://uodo.gov.pl/>
- ISO/IEC 27001:2022 - *Information security, cybersecurity and privacy protection - Information security management systems - Requirements*.
- NIST. *Special Publication 800-57: Recommendation for Key Management*. <https://csrc.nist.gov/publications/sp>
- Prezentacje z wykładów